Faculty of Computer Science and Information Sciences

Scientific Computing Department

Realistic Images Generation

This documentation is submitted as required for the degree of bachelor's in computer and Information Sciences.

By

MennatAllah Wael Saed	[Scientific Computing Department]
Nada Mohamed Taalab Abdullah	[Scientific Computing Department]
Ehab Mohamed Abd Elmonem	[Scientific Computing Department]
Ammar Yasser Mahmoud Saad	[Scientific Computing Department]
Mohamed Hesham Eldesouky	[Scientific Computing Department]
Ahmed Shaaban Abdo Hassan	[Scientific Computing Department]

Under Supervision of

Dr. Manal Mohsen Tantawi

Associate Professor at Scientific Computing Department,
Faculty of Computer and Information Sciences,
Ain Shams University.

TA. Radwa Reda Hossieny

Teaching Assistant of Scientific Computing Department,
Faculty of Computer and Information Sciences,
Ain Shams University.

Cairo
July 2024

Faculty of Computer Science and Information Sciences

Scientific Computing Department

Realistic Images Generation

This documentation is submitted as required for the degree of bachelor's in computer and Information Sciences.

By

MennatAllah Wael Saed	[Scientific Computing Department]
Nada Mohamed Taalab Abdullah	[Scientific Computing Department]
Ehab Mohamed Abd Elmonem	[Scientific Computing Department]
Ammar Yasser Mahmoud Saad	[Scientific Computing Department]
Mohamed Hesham Eldesouky	[Scientific Computing Department]
Ahmed Shaaban Abdo Hassan	[Scientific Computing Department]

Under Supervision of

Dr. Manal Mohsen Tantawi

Associate Professor at Scientific Computing Department,
Faculty of Computer and Information Sciences,
Ain Shams University.

TA. Radwa Reda Hossieny

Teaching Assistant of Scientific Computing Department,
Faculty of Computer and Information Sciences,
Ain Shams University.

Cairo
July 2024

Acknowledgments

All praise and thanks to ALLAH, who provided the ability to complete this work. we hope to accept this work from me.

We're grateful to our parents and our families who are always providing help and support throughout the whole years of study. we hope we can give that back to them.

We also offer our sincerest gratitude to my supervisor, Dr. Manal Mohsen Tantawy. T.A Radwa Reda Hossieny. who have supported me throughout our work with their patience, knowledge, and experience.

Finally, I would like to thank my friends and all the people who gave me support and encouragement.

Abstract

Recently, Designing has been a very important process in any industry starting from digital marketing to the software industry to the car industry. The problem here is that designing an object or a product has always been a time and resource-consuming task, taking logo design as an example, for a beginner designer it may take two to four days to design one logo. It may take weeks or even months which is too challenging in our rapidly growing world. This requires new products, new technologies, and new ideas every day to cope with the market growth.

In this work, the proposed system can help in the design process using deep learning models, especially generative models which are very suitable to help in this case. Generative models are a special type of deep learning model that can generate images. Some generative models can be fine-tuned with different conditions like text, semantic maps, key poses, images with damaged parts, voice, or hand-drawn sketches. These conditions can control the generation process to be more related to the needed designs. The proposed system focuses on using free-hand sketches to guide the generation process. The system generates real images out of free-hand sketches which helps get a design that matches the user’s requirements. Gans and ControlNet generative models are used, which they're suitable for the system's requirements. What makes them the most suitable is that they can be fine-tuned and a new condition can be used to influence the generation process to a certain result. The best results were achieved from CGAN with 55.02 Fréchet Inception Distance (FID) using the shoesv2 dataset.

Table of Contents

Acknowledgments ii

Abstract iii

Table of Contents iv

List of Figures vii

List of Tables viii

List of Abbreviations ix

Chapter 1 10

Introduction 10

Chapter 1: Introduction 11

1.1 Problem Definition **11**

1.2 Motivation **12**

1.3 Objective **12**

1.4 Time Plan **13**

1.5 Documentation Organization **13**

Chapter 2: Literature Review 15

2.1 General Overview **16**

2.2 Related Studies **18**

2.3Competitive analysis **20**

Chapter 3: System Architecture 22

3.1 Preprocessing **23**

3.1.1 Image Resizing 24

3.1.2 Image Binarization 25

3.1.3 Image Blurring 26

3.1.4 Image Dilation 27

3.15 Histogram Equalization 28

3.2 Feature Extraction Techniques **29**

3.2.1 Edge Detection **29**

3.2.2 Corner Detection **30**

3.2.3 SIFT **30**

3.2.4 HOG **31**

3.2.5 LBP **31**

Chapter 4: Implementation Techniques 32

4.1 Convolution Neural Networks (CNN) **33**

4.2 Recurrent Neural Networks (RNN) **34**
4.3 Encoder-Decoder **35**
4.4 Diffusion Models 35
4.4.1 Stable Diffusion 36
4.4.2 ControlNet 37
4.5 Generative Adversarial Networks (GANs) **39**
4.5.1 pix2pix Conditional GAN 40
4.5.2 Convolution Patch GAN 41
Chapter 5: Experimental Results 44
5.1 Datasets 44
5.2 Measurement Criteria 44
5.3 Experimental Results 45
5.4 User Interface 46
Chapter 6: Conclusions and Future Work 52
6.1 Conclusions 52
6.2 Future Work 53
References 54

List of Figures

Fig 1.1: Car Generated Design 11

Fig 1.2: Generating a real car image out of a hand-drawn sketch of a car with the same features 12

Fig 1.3: Generating a real image of a land escape with pyramids out of a hand-drawn sketch with the same features 12

Fig 1.4:Project Time Plan 13

Fig 2.1:Generation Models Hierarchy 16

Fig 2.2:Machine Learning Architecture 17

Fig 2.3:Deep Learning Architecture 17

Fig 2.4:ControlNet Model Hierarchy 17

Fig 2.5:GAN Models Hierarchy 17

Fig 3.1:System Architecture 23

Fig 3.2:Image Resizing 24

Fig 3.3:Image Binarization 25

Fig 3.4:Image blurring 26

Fig 3.5:Image Dilation 27

Fig 3.6:Image Histogram Equalization 28

Fig 3.7:Edge Detection 29

Fig 3.8:Corner Detection 30

Fig 3.9:SIFT Key points 30

Fig 3.10:HOG Features 31

Fig 3.11:LBP Features 31

Fig 4.1:Proposed System Architecture 33

Fig 4.2:CNN Simple Architecture 34

Fig 4.3:RNN Architecture 35

Fig 4.4:The Architecture of Autoencoder 35

Fig 4.5:Stable Diffusion Architecture 36

Fig 4.6:(a) Stable Diffusion Denoising U-Net (b) ControlNet Copy 38

Fig 4.7:GAN Simple Architecture 39

Fig 4.8:How GANs Are Trained..... 40

Fig 4.9:PatchGAN Discriminator 42

Fig. 5.1 Samples of QuickDraw dataset……...44

Fig 5.2 Home page……...47

Fig.5.3 Model Selection Screen……...48

Fig. 5.4 Generation screen……...49

Fig.5.5. Online Generation……..50

List of Tables

Table 2.1: A brief survey of the key studies of Image Generation. 21

Table 5.1 ControlNet Experimental Results……..45

Table 5.2 CGANS Experimental Results……46

List of Abbreviations

Abbreviation	Stands for
CNN	Convolutional Neural Network
DL	Deep Learning
ML	Machine Learning
SVM	Support Vector Machine
RNN	Recurrent Neural Network
LSTM	Long-Short Term Memory
ResNet	Residual Network
VAE	Variational Autoencoders
GANs	Generative Adversarial Networks
CGAN	Conditional Generative Adversarial Networks
pix2pix	Pixel to Pixel
FID	Fréchet Inception Distance

Chapter 1
Introduction

Chapter 1: Introduction

In this chapter, providing a comprehensive overview will be laid out of the key aspects of the study. Initially, the problem section will articulate the specific issues the project aims to address, highlighting the challenges faced by individuals and the current limitations of existing assistive technologies. Following this, the objectives section will outline the primary goals of the project, and the scope will define the boundaries and focus areas of the research. Additionally, the significance of the study will be discussed, emphasizing the potential impact and benefits of the developed system. Finally, the structure of the document will be presented, offering a roadmap of the subsequent chapters and their content.

1.1 Problem Definition

Designing is a very important process in any industry starting from digital marketing to the software industry to the car industry. The problem here is that designing an object or a product has always been a time and resource-consuming task, taking logo design as an example, for a beginner designer it may take two to four days to design one logo.

It may take weeks or even months which is too challenging in our rapidly growing world. This rapidly growing world requires new products, new technologies, and new ideas every day to cope with market growth. Getting rid of the concept designing process will not help at all. This will lead to developing products that do not satisfy the requirements and this is the main focus of developing this product [1].



Fig.1.1 Car Generated Design

1.2 Motivation

As mentioned before, the design process consumes a lot of time, resources, and effort, and that motivated us the most to save time and effort taken by designers to design concept art for the industry, saving the cost of products and speeding up the production process.

According to Trulist, the rate of increase in the design field is expected to grow by 3% by the year 2030. Recently, new technologies have been developed, but there is not much research made for them and no clear approvals that can be used to solve such a problem [2].



Fig.1.2 Generating a real car image out of a hand-drawn sketch of a car with the same features.

1.3 Objective

The project target is to develop a system that generates images from free-hand sketches using deep learning generative models to make the concept design process faster and less costly.



Fig 1.3 Generating a real image of a land escape with pyramids out of a hand-drawn sketch with the same features.

1.4 Time Plan

The deadlines were met in time and the schedule was very flexible and organized giving us the space to complete each task without any delay.

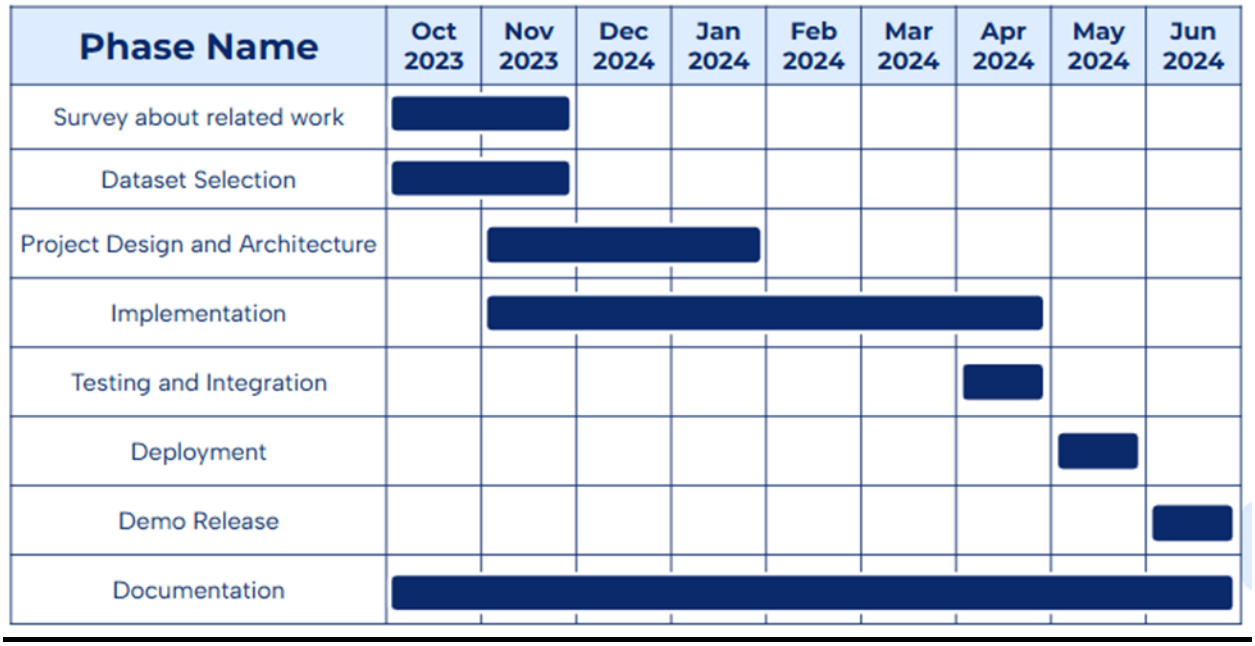


Fig.1.4 Project Time Plan

1.5 Documentation organization

The Project includes six chapters, that's the first. the chapters' description is presented briefly as follows:

Chapter 2 (Related work): This chapter presents a review of previous papers and projects that address similar topics as our project. It discusses the methodologies and results of these prior attempts, providing context and background for our work.

Chapter 3 (System Architecture): This chapter outlines the preprocessing techniques used in our system and their impact on the performance of the models. It details how each technique contributes to improving the overall results of the system.

Chapter 4 (System Implementation): In this chapter, we discuss the various deep learning (DL) models employed in the project. It includes an overview of their architectures and a detailed account of their experimental results.

Chapter 5 (Experimental Result & User Interface): This chapter presents the experimental results for all phases of the system. It also showcases the design of the system's user interface, describing how users interact with the functionalities provided.

Chapter 6 (Conclusion and Future Work): The final chapter summarizes the key findings and contributions of the project. It also outlines potential future work, suggesting improvements and additional features that could further enhance the system's effectiveness.

Chapter 2
Literature Review

Chapter 2: Literature Review

Before starting the project, you should have seen what the researchers had done before. We then scan the papers that address our interests to see what techniques and methods researchers have used in dealing with related projects, so we read many papers dealing with face detection, face recognition, and coin recognition.

In this chapter, we discuss some papers following the general system architecture and present the parts of preprocessing, used models, and results mentioned in each paper. For each paper, we show what filters and sizes were used, what generative algorithms were applied, and what accuracy and processing time the researchers obtained in the paper.

2.1 General Overview

Artificial Intelligence (AI) is an evolving field that focuses on creating intelligent machines capable of performing tasks that typically require human intelligence. It involves the development of algorithms and models that enable machines to learn from data, make decisions, and adapt to new information. AI encompasses various subfields, such as machine learning and deep learning and all related work is recent [3].

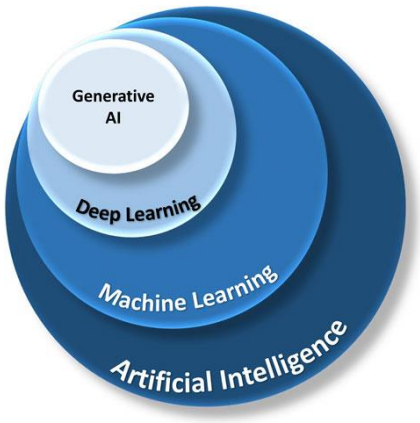


Fig 2 .1 Generation Models Hierarchy

Machine learning is a subset of artificial intelligence (AI) that focuses on using data and algorithms to mimic human learning and improve accuracy over time. In this approach, a machine learning model is trained on a dataset to recognize patterns by utilizing a reasoning algorithm. The traditional machine learning model follows a three-step process. Firstly, preprocessing cleans the dataset used and improves the accuracy of the ML model, using filters like low pass filter, high pass filter, and bandpass filter. Secondly, feature extraction

selects the most effective features from the dataset using loss functions like zero crossing, mean frequency, root mean square, and standard deviation to minimize error between learning and datasets. Finally, classification employs suitable algorithms like SVM, RF, and LDA to accurately classify the dataset [4].

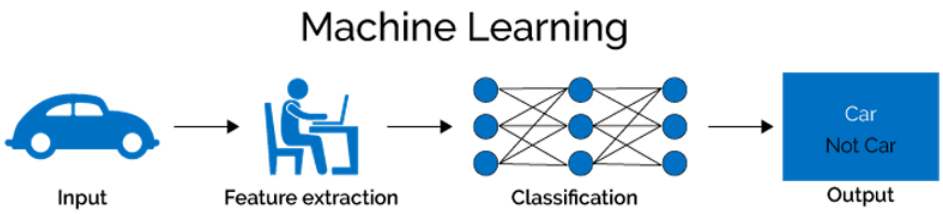


Fig 2.2 Machine Learning Architecture

Deep learning is a subset of machine learning that utilizes artificial neural networks to model and understand complex patterns and relationships in data. It involves training deep neural networks with multiple layers to automatically extract features without exterior feature extraction from raw input. It relies on vast amounts of labeled data and computational power to train deep neural networks, making it suitable for tasks such as image classification and language translation [5].

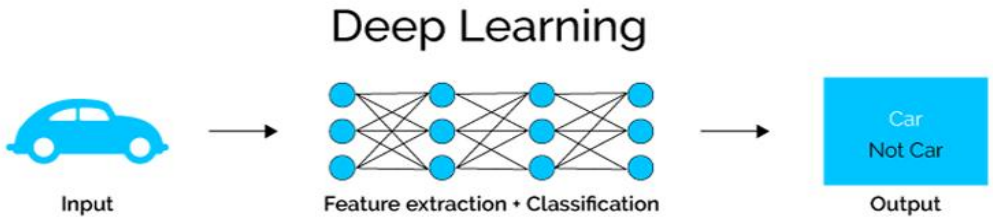


Fig 2.3 Deep Learning Architecture

Generative AI's role in generating images from sketches involves complex algorithms known as neural networks. These networks are trained on large datasets containing pairs of sketches and corresponding images. The training process enables the AI to learn how to interpret the lines and shapes in a sketch and then fill in details, textures, and colors to produce a realistic image. This technology is particularly useful for artists, designers, and anyone looking to quickly bring their ideas to life without needing detailed drawing skills. It's a powerful tool for creative expression and practical applications like product design, animation, and game development.

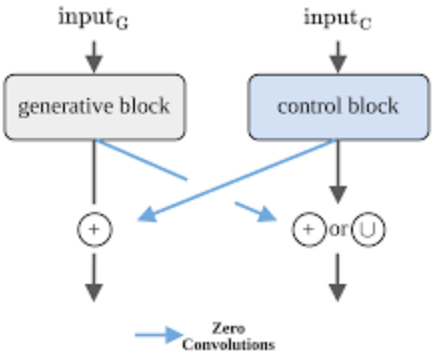


Fig 2.4 ControlNet Model Hierarchy

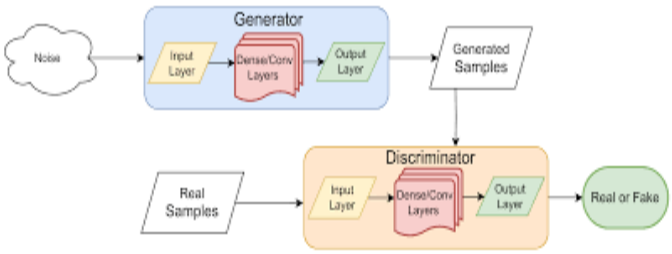


Fig 2.5 GAN Models Hierarchy

2.2 Related Studies

Zhu et al. [6] demonstrate that GANs can be constrained by sketch and color strokes. However, our architecture is a feed-forward mapping from sketch and color to images while Zhu et al. perform an optimization to map user sketches into the latent GAN space to find the most similar image on the natural image manifold (as understood by the GAN). Their approach does not see user inputs at training time and thus cannot learn the complex mapping between user inputs and desired image outputs. Their method is also significantly slower because it is not a strictly feed-forward process and this hinders interactive image editing.

Isola et al. [7] used both approaches to use conditional GANs for the sketch to photo as well as grayscale-to-color synthesis tasks, although they don't focus on user control of the synthesis. The contributions of this paper include: Firstly, and foremost, we are the first to demonstrate an adversarial deep architecture that can learn to generate realistic images from imperfect sketches with sparse color 'scribbles'. Our feed-forward architecture is fast and interactive. Additionally, improves the quality of sketch-to-image synthesis compared to existing work.

Katherine Crowson et al. [8] used VQ-GAN (broader cross-domain model) and CLIP-based optimization of latent vectors to enable text-guided editing of arbitrary images. The model can perform local color and texture changes as well as broad stylistic changes that maintain pixel-level details (e.g. changing the lighting). It cannot perform global changes that alter pixel structure completely but maintain semantic details. For localized object replacement, it can only operate within a predefined mask.

Omri Avrahami et al. [9] used a noisy version of the base image is easier to migrate to larger models but also faces challenges: since the model has never seen the user image, the noising process cannot recover missing details making it hard to balance between fidelity and adherence to the user edit prompt (see figure 10, leftmost column). To combat this, these methods usually use predefined edit masks that limit the edited area throughout the

sampling process. This mitigates the problem but it still exists within the edited portion. Moreover, it's sometimes hard to predict the size of the desired edit area.

Amir Hertz et al. [10] achieved high-quality editing capabilities by operating on the outputs of a text-to-image model. This allows using visual-semantic information encoded in its intermediate attention matrices. Some limitations of relying on the attention weights are that it requires the image to have been generated by the diffusion model, to begin with (they experimented with inverting arbitrary images using DDIM noising with mixed results), requires a base prompt, and only supports a limited set of editing operations (adding or changing a word).

Olaf Ranneberger et al. [11] used the diffusion models to perform the edit operation on the base image. The training data is the image itself (in different crops and augmentations), and the loss function is CLIP-based. The model excels on local, pixel structure preserving changes like adding effects (fire, smoke) or changing texture and color, but lacks visual and semantic richness in its training data which means it cannot perform complex operations like integrating complex objects into the scene or make global changes that maintain semantic details.

Hertzmann et al. [12] used transfer artistic style using pixel features and their local neighborhoods. While more recent algorithms using deep neural networks generate better quality results from a stylistic perspective, this technique allows users to Synthesize new images based on simple annotations. As for recent work on style transfer, it can be split into two categories: specialized algorithms or more general neural approaches.

Bethge et al. [13] used "Gram Matrices" to represent global statistics about the image based on output from convolution layers. These statistics are computed by taking the inner product of in-terminate activations a tensor operation that results in an $N \times N$ matrix for each layer of N channels. During this operation, all local information about pixels is lost, and only correlations between the different channel activations remain. When glitches occur, it's most often due to these global statistics being imposed onto the target image regardless of its statistical distribution, and without any understanding of the local pixel context.

Li and Wand [14] also operated on the output of convolution layers. For layers of N channels, neural patches of 3×3 are matched between the style and content image using a nearest neighbor calculation. Operating on patches in such a way gives the algorithm a local understanding of the patterns in the image, which overall improves the precision of the style transfer since fewer errors are introduced by globally enforcing statistical distributions. Both gram- and patch-based approaches struggle to provide reliable user controls to help

address glitches. The primary parameter exposed is a weighting factor between style and content; adjusting this results in either an abstract-styled mashup that mostly ignores the input content image or the content appears clearly but its texture looks washed out.

2.3 Competitive analysis

A summarized comparison between the key recent studies in our research area is shown in each phase Table, in terms of five main criteria: The Author, Dataset, Preprocessing, Models, and Evaluation.

Table 2.1 Comparison between the Related Studies

Author	Dataset	Preprocessing	Model	Evaluation
Chengying Gao et al. [2020]	SketchyCOCO	Edge Detection	EdgeGAN	FID: 87.6
Xiaoyu Xiang et al. [2021]	• Scribble QMUL-Sketch • SketchyCOCO	____	AODA	FID: 114.8
Changing Gao [2020]	Sketchy-COCO	• Sketch Segmentation • Edge extraction • Attribute vector Generation • Resized	Edge GAN	FID = 80.3
Runtao Liu [2020]	• ShoeV2 • ChairV2	Resized	2-stage model	FID = 48.73
Subhadeep Koley et al. [2023]	• ShoeV2 • Handbags • ChairV2	• Inpainting Enhancements • Style-transfer • Simple Arithmetic Operations	StyleGAN	FID=35.85 FID=100.23 FID=90.21

Ankan Kumar Bhunia et al. [2022]	• Creative Birds • Creative Creatures	• Resizing • Normalization	Doodler Former	FID =16.45 FID =18.71

According to the current studies in the literature presented in the previous section, the following observations can be drawn:

- The most common architectures for this task are GANs.
- GANs are limited to specific domains which gives an advantage to Diffusion Models.
- Diffusion Models are more generalized which makes them suitable for non-domain-specific problems.
- Fréchet inception distance (FID score) score is the common measurement.
- Normalization and Resizing are necessary preprocessing steps.

Chapter 3
System Architecture

Chapter 3: System Architecture

In this chapter, we will explain the system architecture for a Doodle-to-Image Generator using Deep Learning Techniques. The system consists of multiple stages, each incorporating various techniques and algorithms. We will describe each stage, its benefits, and the most popular algorithms that can be applied.

The system architecture includes five stages: Preprocessing, Generation, Post-Processing, and Software Application. The system architecture of the Doodle-to-Image Generator using deep learning is crucial for achieving accurate and efficient generation of realistic images from sketches. This chapter presents an overview of the architectural design, key components, and data flow of the system. The architecture leverages the power of deep learning algorithms to enhance the image generation capabilities of the system. As shown in Fig 3.1, the architecture of the project.

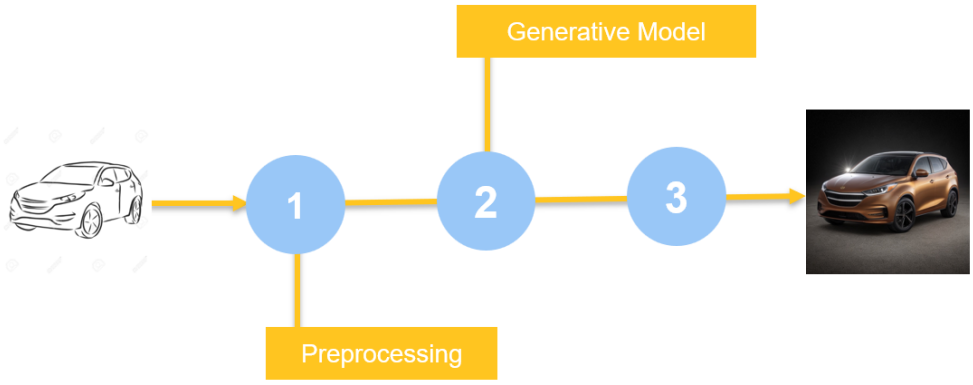


Fig.3.1 System Architecture

3.1 Preprocessing

The Preprocessing component is tasked with preparing the acquired chest radiographic images for subsequent analysis. It performs various operations such as image resizing, noise reduction, contrast enhancement, and normalization. These preprocessing techniques improve the quality and standardization of the input images, enabling better accuracy during the subsequent stages of the system. Common preprocessing methods include resizing to a standard resolution, Binarization, applying filters for noise reduction, and adjusting brightness and contrast levels.

3.2.1 Image Resizing

Image resizing is a fundamental step in the preprocessing stage of the Doodle-to-Image Generator system. et al. [15] This process involves altering the dimensions of input sketches to ensure uniformity and compatibility with subsequent stages of the deep learning pipeline. In this system, all input doodles are resized to a standard size of 500x500 pixels. This resizing has several benefits:

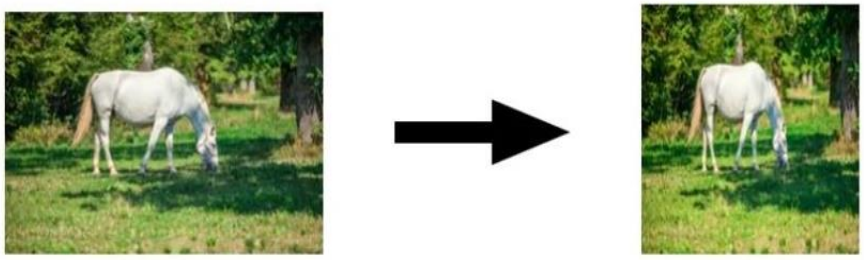


Fig.3.2 Image Resizing

Benefits of Image Resizing:

- Uniform Input Size**:** By resizing all input images to 500x500 pixels, the system ensures that every image has a consistent size, which simplifies the processing pipeline and reduces the complexity of handling varying image dimensions.

- Optimized Memory Usage**:** Standardizing the image size helps in optimizing memory usage, as the system can allocate a fixed amount of memory for each image. This is particularly important when dealing with large datasets and ensures efficient memory management.

- Enhanced Model Performance**:** Many deep learning models perform better with fixed-size inputs. Resizing images to 500x500 pixels ensures that the model receives inputs of a uniform size, which can improve the accuracy and performance of the image generation process.

- Speed and Efficiency**:** Fixed-size inputs allow for more efficient batch processing and parallel computation, leading to faster training and inference times.

3.2.2 Image Binarization

Binarization is a crucial step in the preprocessing stage of the Doodle-to-Image Generator system. et al. [16] This process involves converting grayscale or color images into binary images, where each pixel is either black or white. This simplification can help the system focus on the essential structure of the doodles, enhancing the performance of the deep learning algorithms used in later stages. Here, we discuss the benefits, techniques, and implementation of binarization.

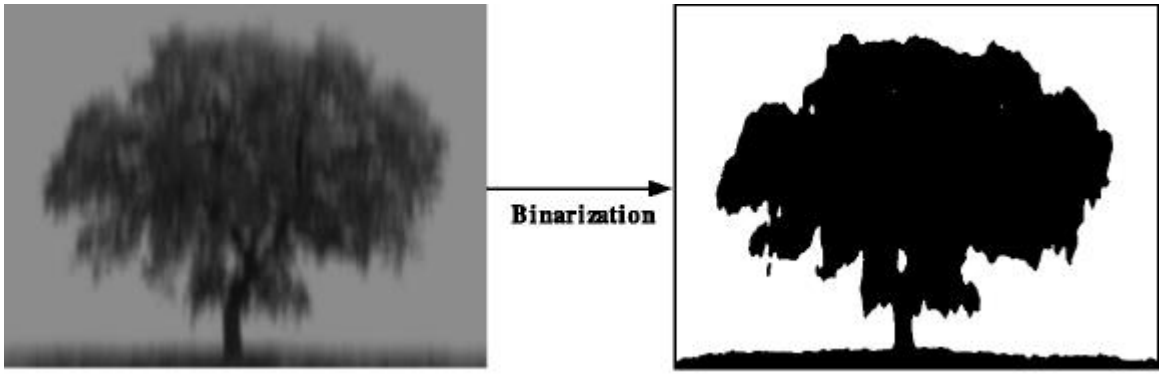


Fig.3.3 Image Binarization

Benefits of Binarization:

- Noise Reduction: Binarization helps in reducing the noise in images by converting all pixels to either black or white. This makes it easier for the system to identify and focus on the primary features of the doodles.
- Simplified Data Representation: Binary images require less storage space and computational power compared to grayscale or color images, leading to faster processing times and more efficient use of resources.
- Enhanced Feature Extraction: With binarized images, the key features of the doodles become more prominent, facilitating more accurate and effective feature extraction by the deep learning models.
- Improved Model Training: Binarization can help improve the training of deep learning models by reducing the complexity of the input data. This allows the models to learn more effectively and generalize better to new sketches.

3.2.3 Image Blurring

Blurring is a crucial step in the preprocessing stage of the Doodle-to-Image Generator system. et al. [17] This process involves smoothing out the details of an image by averaging the pixel values with their neighbors, which helps in reducing noise and irrelevant details. Blurring can enhance the quality of feature extraction and improve the performance of deep-learning models in generating realistic images from sketches. In this section, we discuss the benefits, techniques, and implementation of blurring.

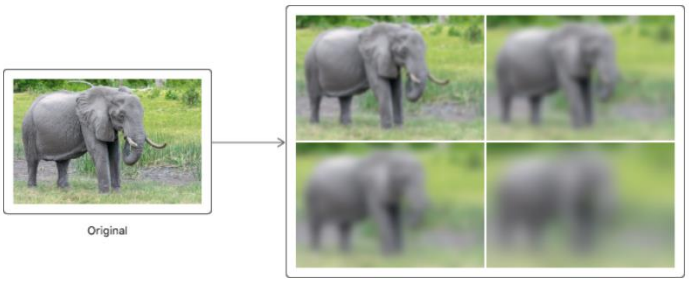


Fig.3.4 Image Blurring

Benefits of Blurring

- Noise Reduction: Blurring effectively reduces noise and minor variations in the input doodles, making the main features more prominent and easier for the system to identify. This is particularly useful for doodles drawn with inconsistent lines or those with stray marks.
- Smoothing Edges: By softening the edges and reducing sharp transitions, blurring helps in creating more natural and continuous outlines, which can be beneficial for generating realistic images. This smoothing effect helps in mitigating the harsh edges that are often present in hand-drawn sketches.
- Improved Feature Extraction: Blurred images can simplify the extraction of important features by removing unnecessary details that could confuse the deep learning models. This allows the models to focus on the primary structure and form of the doodles.
- Enhanced Model Performance: By providing cleaner and more uniform input data, blurring can help in improving the accuracy and robustness of the deep learning models used in the image generation process. This results in a more efficient training process and potentially higher-quality outputs.

3.2.4 Image Dilation

Dilation is an important step in the preprocessing stage of the Doodle-to-Image Generator system. et al. [18] This process involves expanding the boundaries of objects in an image, which can enhance the structural elements and fill in small gaps or holes within the sketches. Dilation helps in emphasizing the primary features of the doodles, making them more prominent and easier for the deep-learning models to interpret. In this section, we discuss the benefits, techniques, and implementation of dilation.

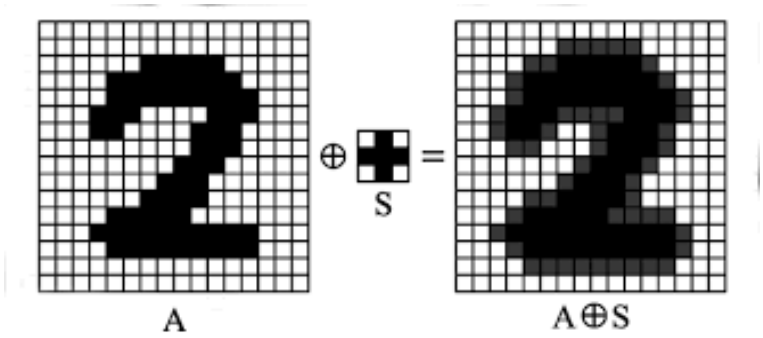


Fig.3.5 Image Dilation

Benefits of Dilation

- Enhancement of Features: Dilation emphasizes the main features of the doodles by expanding their boundaries. This makes the primary structures of the sketches more prominent, which is beneficial for feature extraction and subsequent processing stages.
- Filling Small Gaps: Dilation helps in filling small gaps and holes within the doodles. This can be particularly useful for incomplete sketches or those with broken lines, ensuring that the key elements are well-connected and more easily interpreted by the deep-learning models.
- Noise Reduction: By expanding the boundaries of objects, dilation can help in reducing noise by smoothing out small, isolated points that may be present in the sketches. This results in cleaner and more coherent input data.
- Improved Model Training: Dilation provides clearer and more defined input images, which can enhance the training process of deep learning models. Well-defined features allow the models to learn more effectively and improve their ability to generate high-quality images from doodles

3.1.5 Histogram Equalization

Histogram equalization is a powerful preprocessing technique used to enhance the contrast of images. et al. [19] It works by redistributing the intensity values of the image so that they span the entire range of possible values more evenly. This process improves the visibility of details in both bright and dark areas, making it an essential step for enhancing the quality of input images for deep learning models. In this section, we discuss the benefits, techniques, and implementation of histogram equalization.

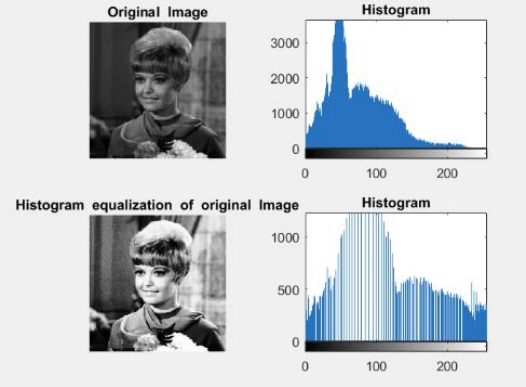


Fig.3.6 Image Histogram Equalization

Benefits of Histogram Equalization

- Improved Contrast: Histogram equalization enhances the contrast of an image, making details more distinguishable. This is particularly useful for images with poor lighting conditions or low contrast.
- Enhanced Feature Visibility: By spreading out the most frequent intensity values, histogram equalization makes important features more visible and prominent. This can lead to better feature extraction and more accurate deep-learning model performance.
- Balanced Intensity Distribution: This technique balances the distribution of pixel intensities, ensuring that the image utilizes the full range of intensity values. This results in a more uniform and visually appealing image.
- Better Model Performance: Preprocessing images with histogram equalization can improve the performance of deep learning models by providing inputs with enhanced and more uniform contrast, leading to more effective learning and generalization.

3.2 Feature Extraction

Feature extraction is a crucial stage in the Doodle-to-Image Generator system, where relevant attributes or characteristics are identified and extracted from input images. These features are then used by deep learning models to generate realistic images from doodles. In this section, we discuss several commonly used feature extraction techniques, their benefits, and references.

3.2.1 Edge Detection

et al. [19] Edge detection techniques identify significant changes in brightness, often corresponding to object boundaries, it's benefits: Highlights important structures and shapes. Simplifies the image by reducing the amount of data while preserving essential features. Enhances model performance by focusing on key elements. There are Common Algorithms:

- Canny Edge Detector**:** Uses a multi-stage algorithm to detect a wide range of edges in images.
- Sobel Operator**:** Computes the gradient of the image intensity, highlighting regions with high spatial frequency.
- Laplacian of Gaussian (LoG)**:** Combines Gaussian smoothing with the Laplacian operator to detect edges.



Fig.3.7 Edge Detection

3.2.2 Corner Detection

et al. [21] Corner detection techniques identify points in the image where the intensity changes sharply in multiple directions, it's benefits: Detects key points of interest that are invariant to rotation and scale, provides robust features for matching and recognition tasks, Enhances the model's ability to understand and reconstruct complex structures, there are Common Algorithms as:

- Harris Corner Detector: Measures the difference in intensity for shifts in various directions to find corners.
- Shi-Tomasi Corner Detector: An improved version of the Harris detector that considers smaller eigenvalues for better accuracy.

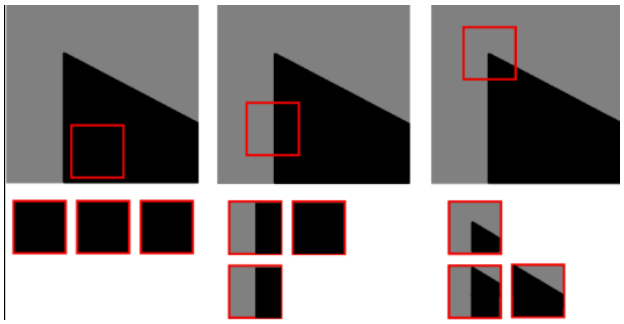


Fig.3.8 Corner Detection

3.2.3 Scale-Invariant Feature Transform (SIFT)

et al. [22] SIFT detects and describes local features in images that are invariant to scaling, rotation, and translation, it's benefits: Provides robust feature descriptors for matching across different images, captures distinctive key points and their local neighborhoods, Improves the system's ability to generate consistent and accurate.

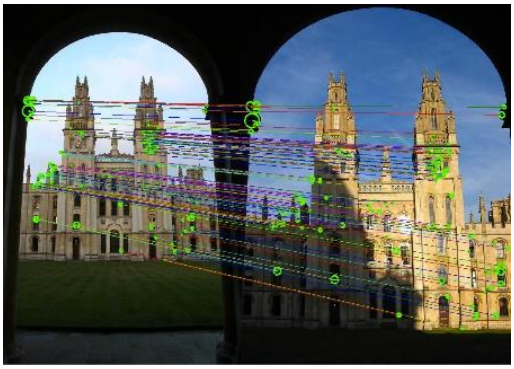


Fig.3.9 SIFT Key points

3.2.4 Histogram of Oriented Gradients (HOG)

et al. [23] HOG computes the distribution of gradient orientations within localized portions of an image, it's benefits: Captures edge and shape information effectively, provides a robust feature representation for object detection and recognition, and enhances model performance by focusing on structural information.

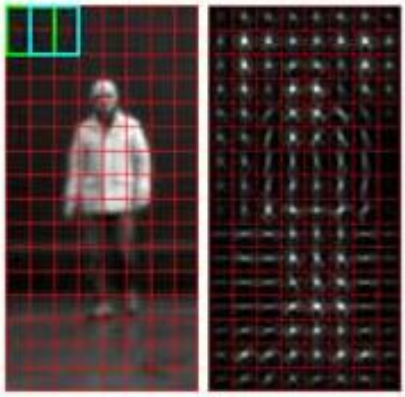


Fig.3.10 HOG Features

3.2.5 Local Binary Patterns (LBP)

et al. [24] LBP captures the texture information of an image by comparing each pixel with its neighbors and encoding the result as a binary pattern, it's benefits are efficient and simple to compute, providing robust texture features for image analysis, and enhancing model performance by capturing fine-grained details.

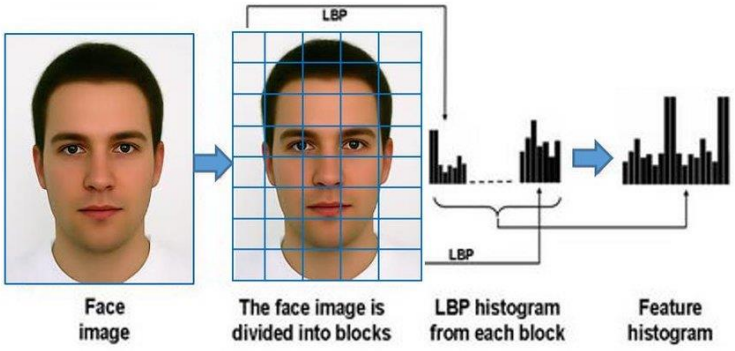


Fig.3.11 LBP Features

Chapter 4
Implementation Techniques

Chapter 4: Implementation Techniques

In this chapter, we discuss the implementation details of the system. The main phase of our system is model selection and training, we show the details of the deep learning approaches and generative methodologies used in the proposed system.

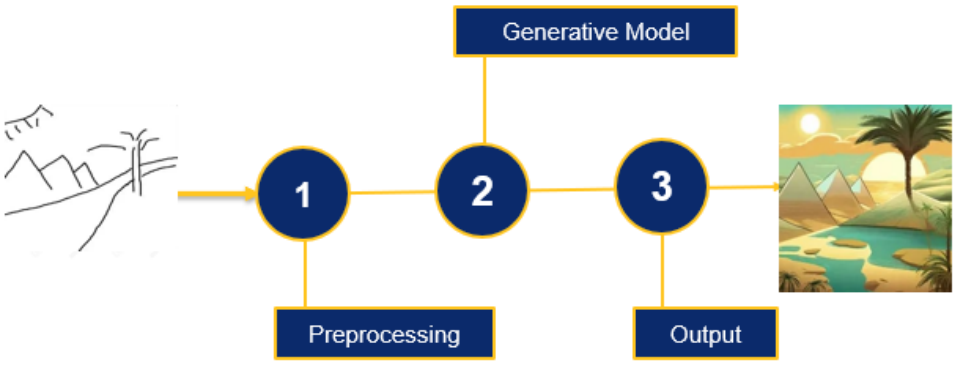


Fig.4.1 Proposed System Architecture

Deep learning approaches have shown promising results in self-supervised tasks such as generative tasks, these results came out of the need to increase data in fields where there's a lack of data or to simulate human tasks, in our task, we generate realistic images based on input sketches, this is an Image-to-Image translation task where the input and the output is a condition in shape of image, such as image segmentation, super-resolution, denoising and generation, the most abstract type of models and the one that all used techniques are built based upon is encoder-decoder models.

Generating realistic images from sketches involves sophisticated deep-learning models that can effectively learn the mapping from a simple line drawing to a detailed and colorful image. This process requires models that can understand the structural, textural, and color details necessary to create a photo-realistic image from a basic sketch. Below are some of the most commonly used deep learning models for this task, along with their benefits and notable applications.

4.1 Convolution Neural Networks (CNN)

CNNs are deep learning models specifically designed for processing grid-like data such as images. They use layers with convolutional filters that can capture spatial hierarchies in images. Its benefits are: Excellent at capturing local features and patterns in images, can be trained to recognize and generate complex image structures, Versatile, and can be used in conjunction with other models like GANs for enhanced performance. Its applications are: Directly translating sketches to images by learning the mapping through supervised learning and enhancing and refining the output of generative models [37][38].

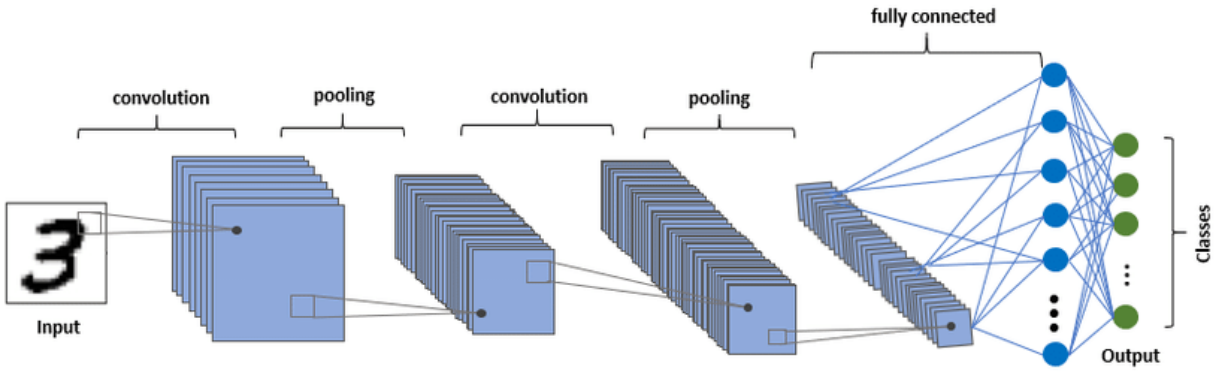


Fig.4.2 CNN Simple Structure

Key Variants:

- U-Net: A CNN architecture that includes an encoder-decoder structure with skip connections, making it suitable for image translation tasks.
- ResNet: Uses residual connections to allow for very deep networks without the vanishing gradient problem.

4.2 Recurrent Neural Networks

While primarily used for sequential data, RNNs and LSTMs can be employed for tasks involving temporal dependencies or progressive image generation. Its benefit is: that it can capture temporal dependencies and long-range context in sequential data and is useful for progressive image generation where the sketch is incrementally refined. Its applications are: Generating images in a step-by-step manner, useful for detailed and complex images, and Enhancing the output of other generative models by iteratively refining the generated image [39].

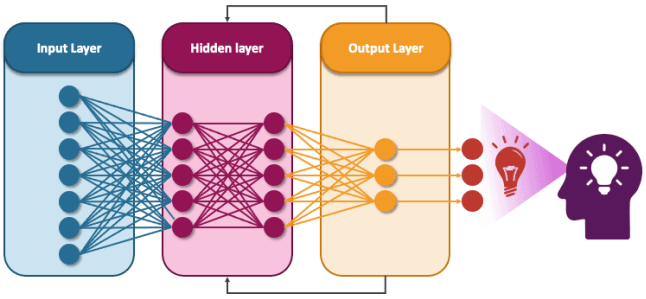


Fig.4.3 RNN Architecture

4.3 Encoder-Decoder

Encoder-decoder architecture (also known as Autoencoder) is a powerful tool for image generation tasks. It functions like a two-part translator. The first part is the encoder, it's a Convolution Neural Network (CNN) consisting of convolution and pooling layers (mostly max pooling), it works as feature extraction and data compression architecture that takes an image and converts it into a simple representation using extracted key features, this representation is known as latent representation, this representation is later fed to the second part which is the decoder, it's also a CNN but works as reconstructor that constructs the images back using the latent representations, it consists of un-pooling and deconvolution layers, Fig.4.1 shows the architecture of the Autoencoder [25][40].

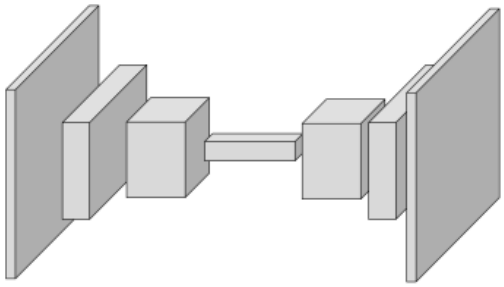


Fig.4.4 The architecture of the Autoencoder

4.4 Diffusion Models

Diffusion models are a special category of probabilistic autoencoders that are used for data generation and denoising [26]. The main functionality is to learn data distribution by repeatedly adding Gaussian noise in the Markov diffusion chain, in each timestep t = 1…T a noisy version of the data (images in our case) is introduced, and each time step is assigned to a specific parameter σ which denotes

the amount of noise to be added at each step, through the reverse diffusion process the model is supposed to repeatedly predict and remove the noise introduced at timestep t = 1…T.

4.4.1 Stable Diffusion

It's an improvement in latent diffusion models and the generative world, it's a condition-based generative model, in basic diffusion models and autoencoders the model learns to generate data by sampling from one distribution, stable diffusion introduces guidance to diffusion models to guide the generative process, the main architecture of diffusion models shown in Fig.4.2 has 3 main parts.

Pixel space is simply an encoder and decoder for images to convert 512x512 images from and to latent representations, the Latent space is where the noise scheduling process is done, instead of performing it in Pixel space and consuming time and memory, the denoising network is a U-Net network, a U-Net is a special type of encoders and decoders based on Fully Convolution Network (FCN) that has symmetrical down-sampling and up-sampling layers making it in the shape of U, symmetrical layers have skip connections between them [28], and lastly, the Conditioning block, which works as an encoder to the condition, the condition received can be text, semantic map, another image and many other conditions[27].

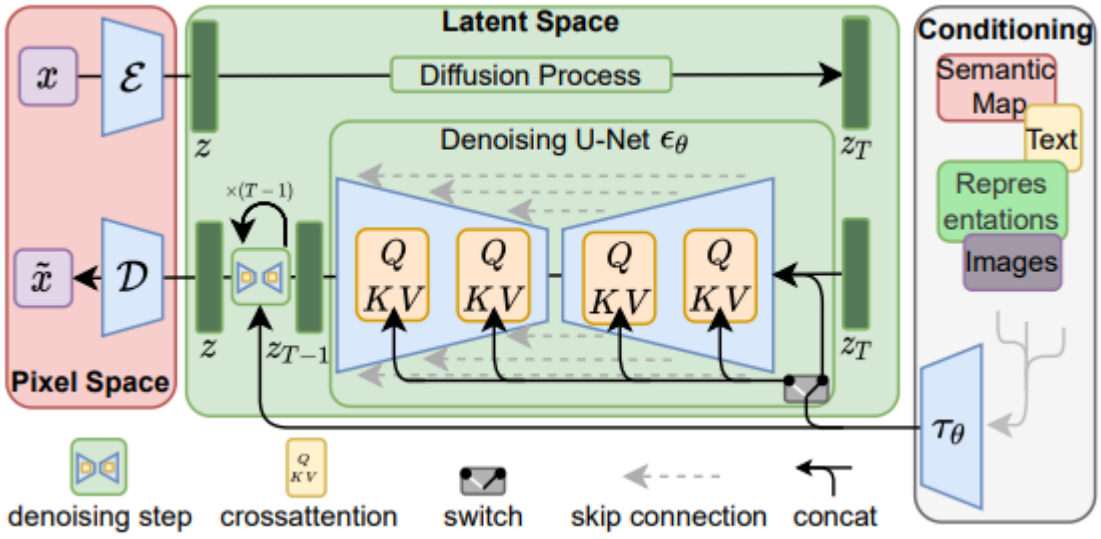


Fig. 4.5 Stable diffusion architecture consisting of 3 parts, Pixel space, Latent space, and Condition encoder.

The input 512x512 image is passed to the Encoder block, a 64x64 latent representation is later fed to the noise scheduler in the diffusion process to introduce the T noisy version, and then the T noisy version is fed to the denoising U-Net concatenated with the encoded condition producing outputs of shapes 32x32, 16x16, and 8x8, then up-sampling these outputs back to produce a new latent representation similar to the original representation of the latent carrying information described in the condition, lastly the new latent of shape 64x64 is fed to the Decoder block to introduce the generated image in 512x512 resolution.

4.4.2 ControlNet

Stable diffusion has shown promising results in text-to-image translation tasks, however, the process of replacing the text encoder with a sketch encoder wasn't possible due to the lack of the type of the condition (sketches) and the huge training time of the stable diffusion, ControlNet has solved these problems introducing a novel adaptive learning method [29], it injects additional condition into the desired block (Denoising U-Net in our case)[20], here we solve the issue of the training time by fine-tuning a pre-trained copy of the Denoising U-Net of the stable diffusion while freezing the original Denoising block of the stable diffusion.

Key Variants of ControlNet

ControlNet variants can vary based on the specific application and integration with different generative models [34][36]:

1. Conditional Generative Adversarial Networks (CGAN):
    - Description: CGANs integrate the input sketch as a condition for the generator to produce realistic images.
    - Benefit: Enables direct control over the generated images based on the input sketch, ensuring fidelity and alignment with the provided visual input[30].

2. Variational Autoencoders (VAEs):
    - Description: VAEs use control signals to guide the latent space representation, enhancing the generation of diverse and realistic images[40].

The input sketch is fed to a zero convolution layer, a zero convolution layer is a convolution layer where all weights and biases are initially zero, then it's concatenated with the input latent of the original U-Net, the ControlNet copies the encoder part from the Denoising U-Net [20] inside the stable diffusion while replacing the decoder part with zero convolution layer and connecting outputs of these layers to the decoder blocks to guide the generation process with our sketch guidance, this will help to protect the backbone of the original Denoising U-Net by eliminating random noise as gradients in the initial training steps, in Figure 4.3 we can see the architecture of the ControlNet along with the Denoising U-Net of the stable diffusion [29].

The ControlNet is trained for 10, 20, and 30 epochs respectively with a learning rate of 0.001 using Adam optimizer and mean square error loss function on a collected dataset of 11 classes each containing 20-23 pairs of images and their corresponding sketches, the input was canny edge detection for the sketches with low and high thresholds of 100, and 200 respectively, the limited size of the data is due to the limited number of proposed sketches data and lack of pairing in terms of positioning, size, orientation … etc.

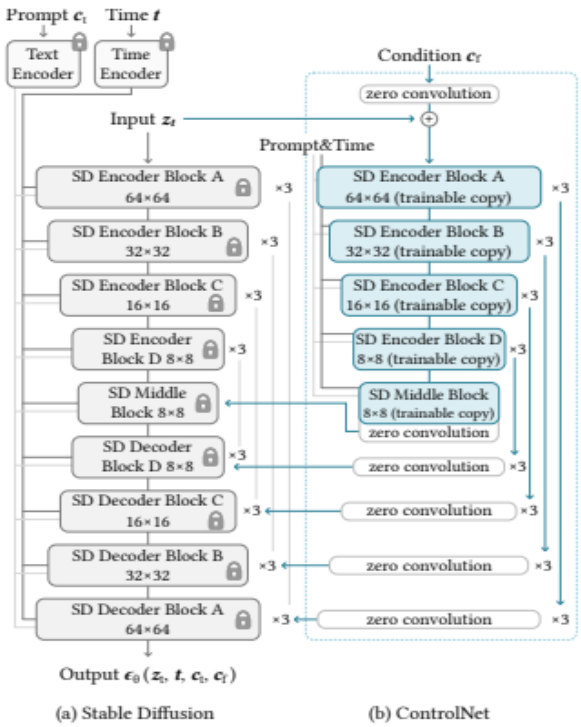


Figure 4.6 (a) stable diffusion denoising U-Net block, gray color, and lock icon represent block freezing (b) the ControlNet copy, the blue part is a fine-tuning part

4.5 Generative Adversarial Networks (GANs)

Another category of the generative models is Generative Adversarial Networks (GANs) [30], they were introduced as a novel generative method, GANs consist of 2 main networks betting against each other, Generator *G* and Discriminator *D*. The generator is the network responsible for the generation task, unlike autoencoders and denoising diffusion models, a simple GAN generator deals with the generation task as a transformation task, where it takes input noise vector *z* and applies various up-sampling and deconvolutions to represent a newly generated data.

Both networks are trained together in a competitive setting. The generator creates images from sketches, while the discriminator evaluates them against real images. its benefits are: Capable of generating high-quality, detailed images, Effective at learning, complex mappings between different image domains, Flexible, and can be adapted for various image translation tasks. Its applications include Image-to-image translation tasks such as converting sketches to photographs, Artistic style transfer, and colorization of black-and-white images[34][35].

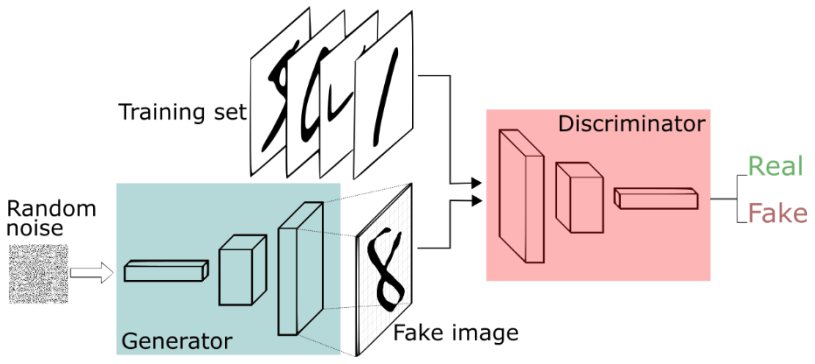
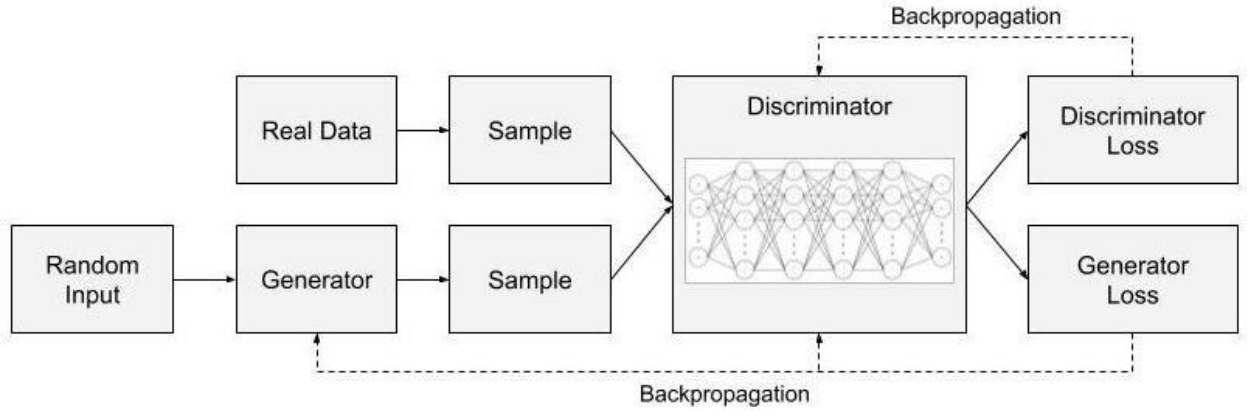


Fig.4.7 GAN Simple Structure

It helps the generator capture the data distribution, while the Discriminator is responsible for estimating whether an input is a real data example or generated from generator *G.*, both are trained simultaneously, and two (adversarial) loss

functions are calculated from the discriminator, one needs to be maximized to increase the generator ability to generate samples similar to the real ones, and the other needs to be minimized to help discriminator discriminate between real and generated samples as shown in Figure 4.4, these models can be used for various



generation tasks such as data generation, data restoration and super-resolution [31].

Figure 4.8 how generative adversarial networks (GANs) are trained, and loss functions are calculated.

GANs Key Variants:

- Pix2Pix: A conditional GAN that uses paired image data to learn the mapping from input sketches to output images.
- CycleGAN: An extension that can learn to translate between two image domains without paired training examples.

4.5.1 pix2pix Conditional GAN

To extend a GAN model to be a conditioned model where the generation task is based on guidance *y* which can be any type of auxiliary information such as a class label or any other type of condition, the condition *y* is fed to both the discriminator and generator as additional input layers, this model is called Conditional GAN (CGAN)[30].

As our task is image-to-image translation, the generator of the CGAN in our case takes a U-Net [20] architecture, it takes a 256x256 input sketch, then it's passed through 7 down-sampling layers consisting of conv layer with a stride of 2

and padding of 1, number of filters are 64, 128, 256, 512 respectively and 512 is repeated through the last 4 layers with Leaky ReLU activation function, producing outputs of shape 128x128, 64x64, 32x32, 16x16, 8x8, 4x4, 2x2, and 1x1 respectively, then this output is passed through 7 symmetric up-sampling layers with skip connections between them introducing a new output image of shape 256x256 and 3 channels (RGB).

This type of CGAN is called pix2pix as it converts an input of type image to an output of type image, in our case it converts input sketch to a realistic image, it's once trained on edges2shoes dataset of shoe images and sketch-like edges, 40k+ image-edge pairs and variations are intraclass[30][41].

Later in another experiment, the model was trained on a dataset consisting of objects for 3 classes (butterfly, car, cat), having 12.5k, 16k, and 12.5k images respectively, the model was conditioned on canny edges of the real images to simulate sketches, this helps in increasing number of classes represented, the model trained with 50 epochs with a learning rate of 0.0002, Adam optimizer and Binary Cross Entropy loss with Logits for the discriminator, then the model is finetuned for 150 more epochs on the image-sketch pairs from the sketchy dataset for the used classes with the same parameters except for the learning rate which is increased to 0.001.

4.5.2 Convolution Patch GAN

The discriminator in our CGAN is a convolution Patch GAN, the idea behind the Patch GAN is that instead of dealing with an image as a whole, we discriminate images in patches or small parts [32], and the discriminator decides whether each patch is part of a real or fake image, then the adversarial loss is calculated from the average losses for each patch, this way increases the generalization ability and enhances the learning process because as the resolution increases we need to pay more attention to details [33].

The input to the Patch GAN discriminator is the concatenation of both the image and edges along the channel axis, the input is passed through a series of convolution layers with different numbers of filters 64, 128, 256, 512, 1 with a stride of 2, and padding of 1, as long as a zero padding layer before the last convolution layer, this introduces inputs of shapes 128x128, 64x64, 32x32, 16x16, 17x17, 16x16 respectively, the last 16x16 output is the patches output, it has a single channel indicating the probability of a patch being real as shown in Figure 4.5 [33].

By combining both the U-Net generator and the Patch GAN discriminator, a special type of conditional GANs is presented to generate high-resolution realistic images from sketch-like edges, also this architecture can be used for similar tasks such as image painting or super-resolution using a more simple and less expensive approach compared to autoencoders and denoising diffusion models.

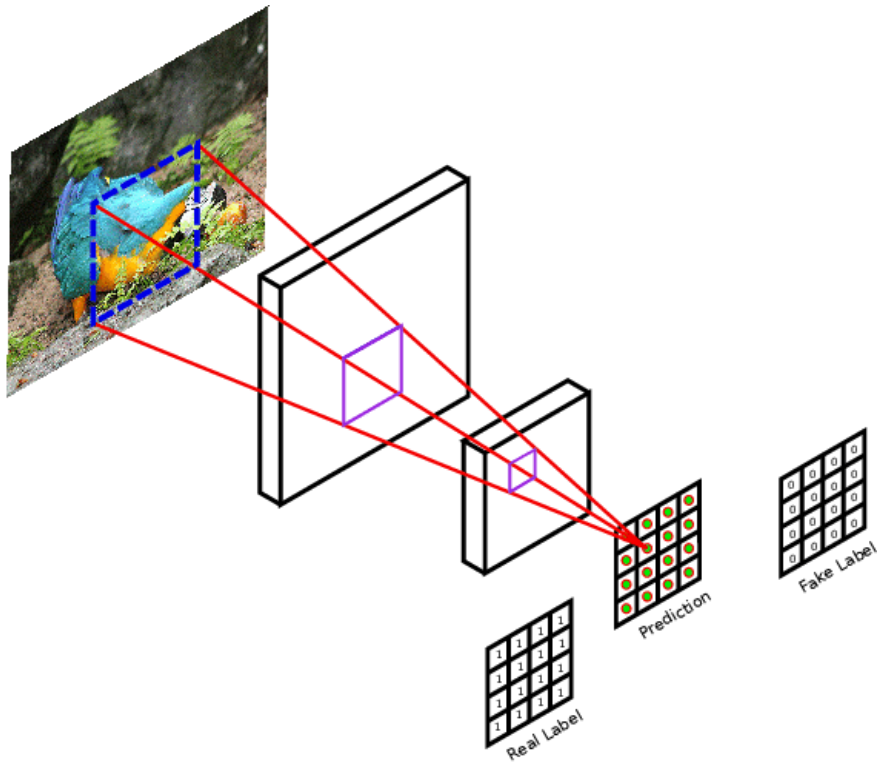


Figure 4.9 Patch-GAN discriminator, the output 16x16 matrix is the probability of each patch being real or fake.

Chapter 5
Experimental Results

Chapter 5: Experimental Results

5.1 Datasets

The datasets used in this experiment consist of images and corresponding free-hand sketches from two sources. Images were collected for 11 different classes, with each class containing 20 images. These images were matched with suitable sketches from the QuickDraw dataset [42].

The Quick Draw Dataset is a collection of 50 million drawings across 345 categories, contributed by players of the game Quick, Draw! The drawings were captured as timestamped vectors, tagged with metadata including what the player was asked to draw and in which country the player was located.



Fig. 5.1 Samples of QuickDraw dataset

5.2 Measurement Criteria

Experiments have been conducted for both recognition and detection approaches. The primary objective of this work is not only to achieve lower Fréchet Inception Distance (FID). The Fréchet Inception Distance (FID) is a widely used metric for assessing the quality of images generated by generative models. It compares the distribution of features extracted from real and generated images using a pre-trained InceptionV3 model [43].

The steps for calculating FID are:

1. Extract features from the real and generated images using the InceptionV3 model.
2. Compute the mean and covariance of these features for both sets of images.
3. Calculate the Fréchet distance between the two multivariate Gaussians defined by these statistics.

The formula for FID is:

$$\mathrm{FID}(x, g) = \|\mu_x - \mu_g\|^2 + \mathrm{Tr}(\Sigma_x + \Sigma_g - 2(\Sigma_x \Sigma_g)^{1/2}) \quad (5.1)$$

where μx and μg are the means, and Σx and Σg are the covariances of the feature representations of the real and generated images, respectively. This metric captures both the quality and diversity of the generated images.

5.3 Experimental Results

This section presents the results obtained using applied models for generating images from free-hand sketches. The primary parameter that varied in these experiments was the number of epochs. The FID scores were used to evaluate the quality of the generated images.

Table 5.1 ControlNet Experimental Results

No. of Epochs	FID Score
10	7996.2983
20	**6487.2587**
30	7550.7021

The results were obtained using Conditional Generative Adversarial Networks (CGANs). Like the experiments with ControlNet, the number of epochs was varied, and the FID scores were recorded to assess performance.

Table 5.2 CGANS Experimental Results

No. of Epochs	FID Score
50	131.04
100	131.78
150	**55.02**
200	213.52

The results indicate the importance of careful tuning of training epochs to achieve optimal performance in both ControlNet and CGANs.

For ControlNet, increasing the number of epochs from 10 to 20 improved the FID score, indicating better image quality, but further increasing the epochs to 30 led to a decline in image quality. For CGANs, the FID scores showed a more stable and predictable trend.

As the number of epochs increased from 50 to 150, FID scores improved significantly, with the best performance at 150 epochs. However, increasing the epochs to 200 resulted in poorer image quality.

Overall, CGANs achieved better performance with lower FID scores, indicating higher-quality generated images compared to ControlNet. This underscores the need for balancing the number of training epochs to optimize image generation quality.

5.4 User Interface

A graphical user interface (GUI) is a visual tool that allows users to engage with electronic devices or software applications. Utilizing graphical components such as icons, buttons, and windows, facilitates the representation and manipulation of data. GUIs improve user experience by offering an intuitive and accessible environment for interaction and navigation.

5.4.1 Main Page

The screenshot depicts the home page of the Imagine-Gen website, which is designed to transform creative sketches into realistic images. The interface is user-

friendly and visually appealing, aimed at guiding users through the process seamlessly.

Key Sections and Functionality: Navigation bar. contains main three links

1. Home: redirect to the home page.
2. About providing data about the team.
3. Contact: offers users a way to get in touch with us.

Get Start button: this button invites users to begin their journey with Imagin-Gen

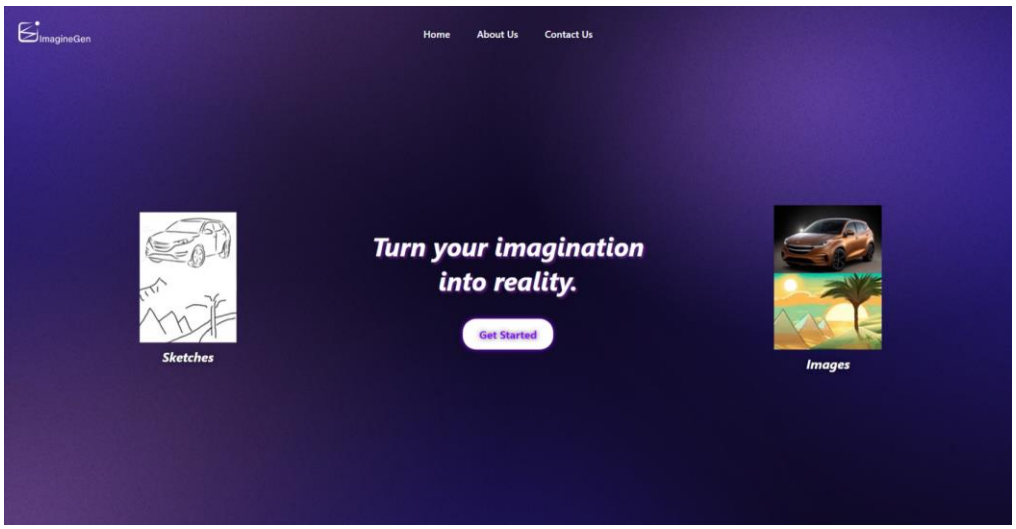


Fig 5.2 Home page

5.4.2 Model Selection Page

The screenshot illustrates the starting page of the Imagine-Gen application, where users can choose their preferred method to begin the transformation process. This interface is designed to be straightforward, guiding users through their options with clear labels and buttons.

Key Sections and Functionality:

1. ControlNet and CGAN options: Users can choose either method based on their preference or familiarity with these techniques.
2. Action Buttons:
   - Sketch: By clicking this button, users can start with a sketch. This led to an interface where users could draw directly the sketches.
   - Upload: By clicking this button, users can upload their sketches or images from their device to begin the transformation process. This

ensures flexibility and convenience for users who already have prepared sketches.

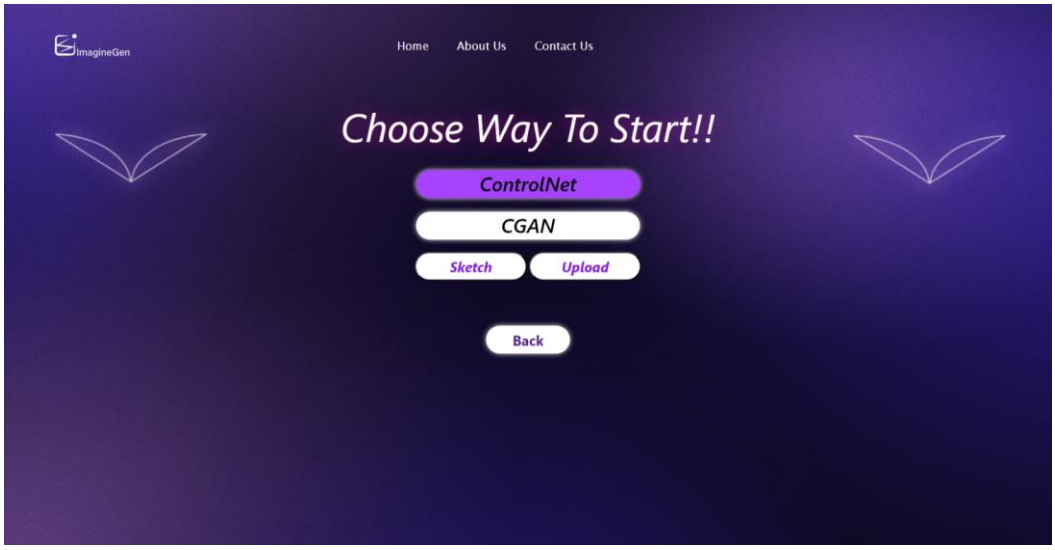


Fig.5.3 Model Selection Screen

5.4.3 Generation page from uploaded sketch

The screenshot shows the "Upload and Generate" page of the Imagine-Gen application. This interface allows users to upload their sketches and generate realistic images from them

Key Sections and Functionality:

1. Upload Sketch Section: Upload Box: Central area on the left side where users can click to upload their sketch.
2. Generated Image Section:
    - Generated Image Box: Positioned on the right side, this area will display the resulting image once generated.
    - Generate Button: Below the image box, clicking this button initiates the process of converting the uploaded sketch into a realistic image.
    - Download Button: Once an image is generated, this button allows users to download the final product

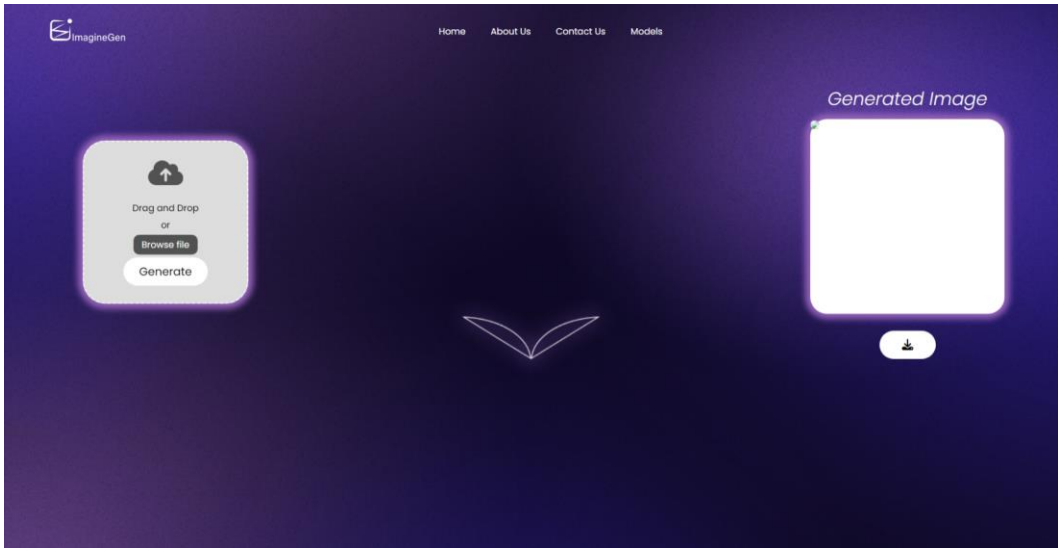


Fig. 5.4 Generation screen

5.4.4 Generation page from sketch

The screenshot shows the "Sketch and Generate" page of the Imagine-Gen application. This interface allows users to draw their sketches and generate realistic images from them

Key Sections and Functionality:

1. Drawing Section: draw Box: Central area on the left side where users can click to draw their sketch, Editing and Management Icons: allow users to modify and download their sketches.
2. Generated Image Section:
    - Generated Image Box: Positioned on the right side, this area will display the resulting image once generated.
    - Generate Button: Below the image box, clicking this button initiates the process of converting the uploaded sketch into a realistic image.

- Download Button: Once an image is generated, this button allows users to download the final product

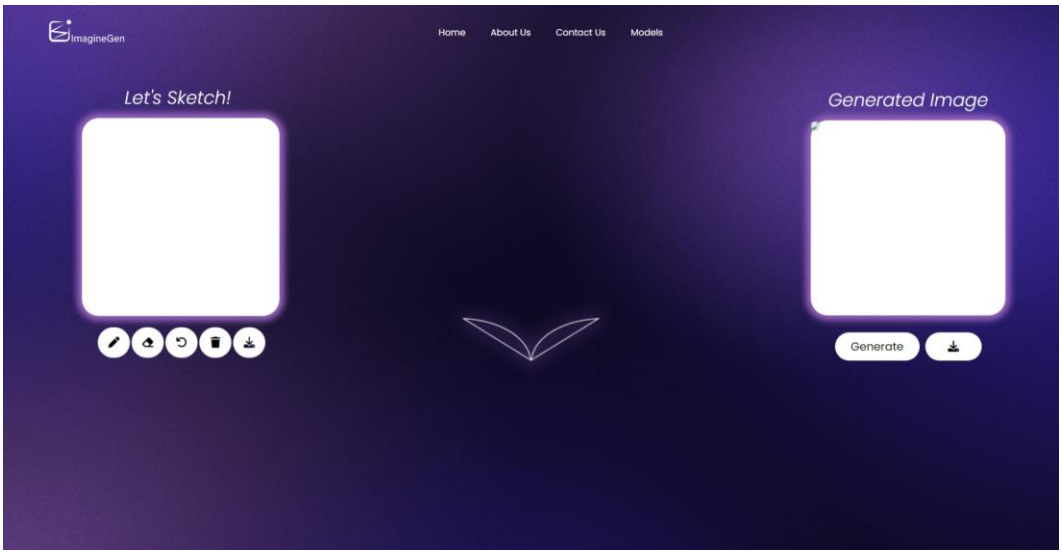


Fig.5.5. Online Generation

Chapter 6
Conclusion and Future Work

Chapter 6: Conclusions and Future Work

In this chapter, we will summarize the problem and present its impact on the community. We will also show how we solved this problem, the models used and the GUI to display to the user. We will also show in future work what extra features can be added together.

6.1 Conclusions

The project goal was to develop a system that generates images from hand sketches using deep learning, making the process of concept design faster and more cost-effective. To achieve this, two separate models are utilized: ControlNet and CGAN.

ControlNet is specialized in transforming hand-drawn sketches into detailed, high-resolution images, ensuring precise control over the generated output. On the other hand, CGAN (Conditional Generative Adversarial Network) generates realistic images directly from sketches, focusing on enhancing the overall quality and realism of the outputs. Each model operates independently, offering unique advantages depending on the specific requirements of the design task. By using these advanced deep-learning models.

ControlNet model, we compiled a dataset from various sources and achieved an FID score of 6487. To ensure the best results, we performed several preprocessing steps including blurring, binarization, and resizing of the images. For the CGAN model, we utilized the pix2pix edge2shoes dataset, with this dataset, we achieved an FID score of 55. Similar to ControlNet, we applied preprocessing techniques such as blurring, binarization, and resizing to enhance the quality and consistency of the training data.

Finally, we developed a clear and simple user interface through which users can select the model they want to use. They also have the option to either draw their sketches by hand or upload an existing sketch from their device. The resulting image generated by the selected model is then displayed to the user, who can download it to their device

6.2 Future Work

In the future, the results can be enhanced by creating a large dataset that includes drawn images versus real images. Increasing the number of images and the diversity of categories will help achieve more accurate and diverse outputs. In the future, we can also work on different pre-processing techniques that will further improve the quality of the images generated.

In addition, new deep learning architectures and fine-tuning strategies can be tried to enhance the performance of the applied models. Incorporating user feedback and constant iteration on the user interface will also be a priority to ensure a more intuitive and efficient user experience. We hope that in the future the design process will be faster, more cost-effective, and accessible to a wider range of users.

References

1. Sheila. 2024 ApexCreative.
2. Branka. 2024 Trulist graphic design statistics.
3. IBM.(n.d). Artificail intellignece. Retrieved from What is Artificial Intelligence (AI)? | *IBM*
4. Coursera. What is machine learning? Mar 27, 2024. What Is Machine Learning? Definition, Types, and Examples | *Coursera*
5. Google Cloud. What is deep learning? 2024, What is Deep Learning? Applications & Examples | *Google Cloud*
6. J.-Y. Zhu, P. Krahenb¨uhl, E. Shechtman, and A. A. Efros. 2016 ¨Generative visual manipulation on the natural image manifold. In Proceedings of European Conference on Computer Vision (ECCV).
7. P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros 2016 Image-to-image translation with conditional adversarial networks. arxiv.
8. Katherine Crowson, Stella Biderman, Daniel Kornis, Dashiell Stander, Eric Hallahan, Louis Castricato, and Edward Raff. Vqgan-clip: Open-domain image generation and editing with natural language guidance,2022.
9. Omri Avrahami, Dani Lischinski, and Ohad Fried. Blended diffusion for text-driven editing of natural images, 2021.
10. Amir Hertz, Ron Mokady, Jay Tenenbaum, Kfir Aberman, Yael Pritch, and Daniel Cohen-Or. Prompt-to-prompt image editing with cross-attention control, 2022.
11. Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation, 2015.
12. Hertzmann, A.; Jacobs, C.; Oliver,N.; Curless, B.; and Salesin, D.. Image analogies. Siggraph Conference Proceedings, 2001.
13. Gatys, L. A.; Ecker, A. S.; and Bethge, M. A neural algorithm of artistic style. CoRR abs/1508.06576, 2015.
14. Li, C., and Wand, M. Combining Markov random fields and convolutional neural networks for image synthesis. abs/1601.04589, 2016.
15. Parker, J.R. (2010). *Algorithms for Image Processing and Computer Vision*. Wiley.
16. Gonzalez, R.C., & Woods, R.E. (2008). *Digital Image Processing*. Prentice Hall.
17. Russ, J.C. (2011). *The Image Processing Handbook*. CRC Press.

18. Haralick, R.M., Sternberg, S.R., & Zhuang, X. (1987). "Image Analysis Using Mathematical Morphology". *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-9(4), 532-550.

19. Gonzalez, R.C., & Woods, R.E. (2008). *Digital Image Processing*. Prentice Hall.Soille, P. (1999). *Morphological Image Analysis: Principles and Applications*. Springer-Verlag.

20. Xintao Wang; Yu Li; Honglun Zhang; Ying Shan (2021) "Towards Real-World Blind Face Restoration with Generative Facial Prior". *Applied Research Center (ARC), Tencent PCG*

21. Shi, J., & Tomasi, C. (1994). "Good Features to Track". *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 593-600.

22. Lowe, D.G. (2004). "Distinctive Image Features from Scale-Invariant Keypoints". *International Journal of Computer Vision*, 60(2), 91-110.

23. Dalal, N., & Triggs, B. (2005). "Histograms of Oriented Gradients for Human Detection". *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 886-893.

24. Ojala, T., Pietikäinen, M., & Harwood, D. (1996). "A Comparative Study of Texture Measures with Classification Based on Featured Distributions". *Pattern Recognition*, 29(1), 51-59.

25. Yifei Zhang (2018) "A Better Autoencoder for Image: Convolutional Autoencoder". *Australian National University ACT 2601*

26. Kuang Gong; Keith Johnson; Georges El Fakhri; Quanzheng Li; Tinsu Pan (2023) "PET image denoising based on denoising diffusion probabilistic model". *European Journal of Nuclear Medicine and Molecular Imaging*

27. Robin Rombach; Andreas Blattmann; Dominik Lorenz; Patrick Esser; Bjorn Ommer (2022) " High-Resolution Image Synthesis with Latent Diffusion Models". *Ludwig Maximilian University of Munich & IWR, Heidelberg University, Germany*

28. Jixun Pan (2024) "Image Segmentation Based On U-Net and Adjusted U-Nets". *New York University, New York, United States*

29. Lvmin Zhang; Anyi Rao; Maneesh Agrawala (2023) "Adding Conditional Control to Text-to-Image Diffusion Models". *Stanford University*

30. Mehdi Mirza; Simon Osindero (2014) "Conditional Generative Adversarial Nets". *Departement d'informatique et de recherche operationnelle Universite de Montreal Montreal, QC H3C 3J7; Flickr / Yahoo Inc. San Francisco*

31. *ml-science.com*. Generative Adversarial Networks [Generative Adversarial Networks — The Science of Machine Learning & AI (ml-science.com)]

32. Aniket A. Tolpadi; Johanna Luitjens; Felix G. Gassert; Xiaojuan Li; Thomas M. Link; Sharmila Majumdar; Valentina Pedoia (2023) "Synthetic Inflammation Imaging with

PatchGAN Deep Learning Networks". *Department of Bioengineering, University of California, Berkeley, CA 94720, USA*

33. Ugur Demir; Gozde Unal (2018) "Patch-Based Image Inpainting with Generative Adversarial Networks". *Ankara Yildirim Beyazit University; Istanbul Technical University*
34. Isola, P., Zhu, J.Y., Zhou, T., & Efros, A.A. (2017). "Image-to-Image Translation with Conditional Adversarial Networks". *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).*
35. Zhu, J.Y., Park, T., Isola, P., & Efros, A.A. (2017). "Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks". *Proceedings of the IEEE International Conference on Computer Vision (ICCV).*
36. Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). "Attention is All You Need". *Proceedings of the Advances in Neural Information Processing Systems (NeurIPS).*
37. Ronneberger, O., Fischer, P., & Brox, T. (2015). "U-Net: Convolutional Networks for Biomedical Image Segmentation". *Proceedings of the International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI).*
38. He, K., Zhang, X., Ren, S., & Sun, J. (2016). "Deep Residual Learning for Image Recognition". *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).*
39. Hochreiter, S., & Schmidhuber, J. (1997). "Long Short-Term Memory". *Neural Computation*, 9(8), 1735-1780.
40. Christoph Feichtenhofer; Haoqi Fan; Yanghao Li Kaiming He (2022) "Masked Autoencoders As Spatiotemporal Learners". Meta *AI, FAIR*
41. Ting-Chun Wang; Ming-Yu Liu; Jun-Yan Zhu; Andrew Tao1 Jan Kautz; Bryan Catanzaro (2018) "High-Resolution Image Synthesis and Semantic Manipulation with Conditional GANs". *NVIDIA Corporation UC Berkeley*
42. Google LLC. "Quick, Draw! The Data." Quick, Draw Accessed 13 Jun. 2024 [https://quickdraw.withgoogle.com/data]
43. Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., & Hochreiter, S. GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium. In Advances in Neural Information Processing Systems 30 (pp. 6626-6637). 2017